

Documentação do Trabalho Prático 1 - Cloud

Igor Joaquim da Silva Costa¹ Jorge Augusto de Lima e Silva¹

¹Departamento de Ciência da Computação
Universidade Federal de Minas Gerais (UFMG)

1. Task 1

A estratégia adotada para manter o estado necessário ao cálculo da média móvel do uso de CPU consistiu em armazenar os valores das porcentagens de uso de cada CPU em uma estrutura de dados persistente, dentro do contexto de execução. Esse contexto fornece um ambiente compartilhado, utilizado para manter o histórico de utilização de CPU de forma identificada por uma chave única. A cada invocação da função handler, o código verifica a existência do histórico de cada CPU no ambiente; caso não exista, inicializa-se uma lista vazia. À medida que novos valores de uso de CPU são recebidos, são adicionados à lista correspondente, que é limitada a 60 elementos, mantendo sempre a média dos últimos 60 segundos. A média móvel é calculada pela soma dos valores da lista dividida pelo número de elementos, proporcionando uma visão precisa e atualizada do desempenho das CPUs. Esse mecanismo é essencial para a coleta e processamento contínuo de dados em tempo real, garantindo uma base confiável para monitoramento e análise.

2. Task 2

A Task 2 consistiu no desenvolvimento de um sistema de monitoramento em tempo real, utilizando o Redis para armazenamento de dados e o Dash para visualização das métricas de desempenho. A aplicação exibe dois gráficos principais: um indicador para o uso de memória dedicado ao caching e um gráfico de barras para o uso de CPU por núcleo. As métricas são armazenadas no Redis em formato JSON, permitindo que o painel seja atualizado periodicamente, conforme o intervalo de tempo configurado, oferecendo uma visualização precisa e em tempo real do desempenho do sistema.

A métrica de **uso de memória para caching** é representada por um valor percentual, indicando a quantidade de memória utilizada para armazenar dados temporários. Essa métrica é crucial para avaliar a eficiência da utilização da memória, pois um valor elevado sugere boa otimização dos recursos, enquanto valores baixos podem indicar subutilização da memória. O gráfico de **uso de CPU por núcleo** exibe a porcentagem de utilização de cada núcleo do processador, permitindo identificar sobrecargas e possíveis desequilíbrios na distribuição da carga entre os núcleos.

Essas métricas são coletadas e exibidas em tempo real, possibilitando a rápida identificação de problemas de desempenho, como picos de uso de memória ou CPU. O sistema oferece uma interface simples e eficiente, fornecendo insights valiosos para a gestão de recursos e contribuindo para a manutenção de um ambiente otimizado e com bom desempenho.

Detalhes de Implementação

Tal como na versão original do runtime, toda a configuração do ambiente é realizada por meio de ConfigMaps, com algumas variáveis adicionais:

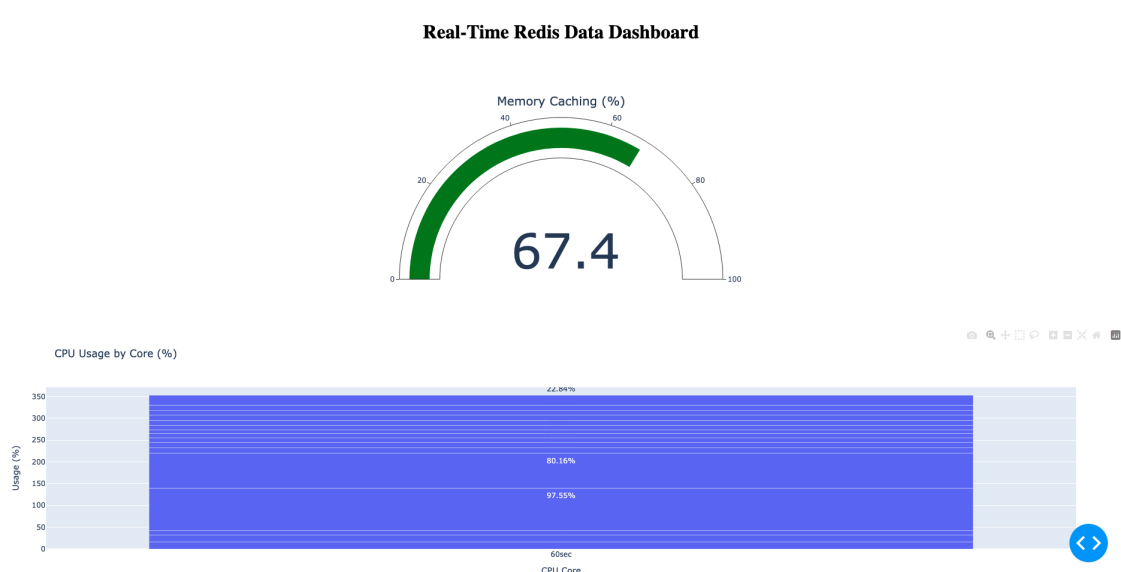


Figure 1. Dashboard de Monitoramento de Desempenho em Tempo Real - Uso de Memória e CPU

- **REDIS_HOST**: o endereço IP do serviço Redis em execução
- **REDIS_PORT**: a porta do serviço Redis em execução
- **REDIS_INPUT_KEY**: a chave consumida pela função lambda
- **REDIS_OUTPUT_KEY**: a chave onde a função lambda escreve o resultado JSON
- **POOL_INTERNAL**: o intervalo (em segundos) no qual a função lambda é executada — valor padrão de 5 segundos
- **ZIP_FILE**: a URL da pasta zip contendo o código a ser executado
- **FUNCTION_HANDLER**: nome da função a ser chamada — handler por padrão

As quatro primeiras variáveis são também utilizadas na versão original do runtime, garantindo compatibilidade entre as implementações. Além disso, o usuário pode configurar a função serverless por meio do `ConfigMap pyfile`, assim como no runtime original.

A versão implementada do runtime inclui algumas extensões, como o suporte a módulos Python complexos via arquivos zip. Quando essa variável está definida, todo o código Python da pasta é carregado em memória, e a função executada será aquela identificada pela variável `FUNCTION_HANDLER`. Se houver uma configuração tanto via `pyfile` quanto um arquivo zip, o código contido em `pyfile` será priorizado.

Dessa forma, a nova implementação do runtime mantém total compatibilidade com a versão original, pois ambos implementam os mesmos contratos e comportamentos semelhantes.